# Stylish Lab Bridebox Salon Management System — Software Requirement Structure

## 1. Project title

**Customizable Small Salon Management System**

## 2. Project overview

This system is designed for a small salon business to manage daily operations, employee performance, customer services, credits, expenses, and business profit tracking. The system must be fully customizable and optimized for both desktop and mobile devices, especially for employee use on smartphones.

The salon owner should be able to monitor the business by **day, week, month, and year**, while employees should be able to log in with restricted access to record services, customers, and selected expenses.

---

## 3. Main objectives

The system should help the salon owner to:

- track total sales and real profit
- manage employees and commission percentages
- monitor employee performance
- manage service packages and pricing
- handle customer credit records
- record business-related expenses
- track bills like rent, electricity, and water
- access the system easily from mobile devices

---

# 4. User roles

## 4.1 Admin / Owner

The owner has full access to the system.

### Admin permissions

- manage salon service packages
- manage employee accounts
- create usernames and passwords for employees
- set and update employee commission percentages
- manage debtors/payees such as suppliers and banks

- view all sales records
- view all employee performance reports
- record owner expenses and monthly bills
- track daily, weekly, monthly, and yearly profit
- manage customers and credit settlements
- view mobile-responsive admin dashboard

---

## 4.2 Employee

Employees have restricted system access.

### Employee permissions

- log in with assigned username and password
- create new customers
- select existing customers
- register completed services
- mark a customer as credit if payment is delayed
- record certain allowed expenses such as supplier/bank payments
- view their own earnings by day, week, month, and year
- update own profile picture

### Employee restrictions

Employees cannot:

- change service prices
- manage other employees
- change commission settings
- access full business reports
- delete important business records unless permitted by admin

---

# 5. Functional requirements

## 5.1 Login and authentication

The system must provide secure login functionality for both owner and employees.

### Features

- username/password login
- role-based access control
- secure password storage
- profile-based dashboard after login

---

## 5.2 Employee management

The owner should be able to register employees and manage their earning percentages.

### Features

- add new employee
- edit employee details
- activate/deactivate employee
- assign username and password
- define employee commission percentage
- update commission percentage later

### Example

- Employee A: 50% employee / 50% owner
- Employee B: 30% employee / 70% owner during training
- Later owner can increase the share

### Important rule

When a commission percentage is changed, old sales records must not be recalculated incorrectly. Therefore, the system should store the commission values used at the moment each sale was created.

---

## 5.3 Service package management

The owner should be able to create and edit service templates.

### Current example packages

- Haircut – 350
- Beardcut – 350
- Hair & Beard – 700
- Hair, Beard & Massage – 900
- Massage – 300
- Facial – 2500
- Cleanup – 2000
- Hair Color Black – 800
- Hair & Beard Black – 1200
- Special Package – 2500

### Features

- add service package
- edit service name and price
- activate/deactivate service
- display service list to employees during sale entry

### Rule

Employees should see service prices but should not be able to edit them unless special permission is granted.

---

## 5.4 Customer management

Employees and owner should be able to manage customer records.

### Features

- register new customer
- store customer name
- store mobile number optionally
- search existing customer
- select existing customer during service entry

### Rules

- customer name is required
- mobile number is optional

---

## 5.5 Service transaction / sales entry

Employees should be able to record completed customer services.

### Workflow

1. employee logs in
2. selects an existing customer or creates new customer
3. selects service package
4. service price is automatically shown
5. payment type is selected
6. sale is saved
7. date and time are automatically recorded
8. employee share and owner share are automatically calculated

### Payment types

- fully paid
- credit / pay later
- partial payment

### Stored sale details

- customer
- employee
- service
- service price snapshot
- commission percentage snapshot
- employee earning amount

- owner earning amount
- payment status
- paid amount
- due amount
- date and time

---

## 5.6 Credit customer management

If a customer does not pay immediately, the service should be recorded as a credit.

### Features

- mark sale as credit
- record due amount
- view all customers with pending balances
- settle credit payments later
- show credit history

### Example

A customer gets haircut and beard service worth 700 and says he will pay later. The employee should save the sale as credit, and the system should show that due amount until payment is settled.

---

## 5.7 Expense management

The system should support both employee-recorded and owner-recorded expenses.

## 5.7.1 Employee expenses

Sometimes the employee pays a supplier or bank collector when the owner is not present.

### Features

- select expense reason
- select payee
- enter amount
- save expense record
- record who paid it
- record date and time automatically

### Common examples

- bank payment
- supplier payment

---

## 5.7.2 Owner expenses

The owner should be able to record fixed and operational expenses.

### Examples

- electricity bill
- water bill
- shop rent
- other direct business expenses

### Features

- record expense
- mark bill as settled
- track expenses monthly
- include expenses in monthly profit calculation

---

## 5.8 Payee / debtor management

The owner should be able to register external parties who receive payments.

### Examples

- suppliers
- banks
- other collectors

### Features

- add payee
- edit payee
- categorize payee type
- use payee during expense entry

---

## 5.9 Reports and analytics

The system should provide business insights for the owner and performance reports for employees.

## 5.9.1 Owner reports

The owner must be able to view:

### Daily

- total sales
- cash received
- credit sales
- employee commissions
- total expenses

- daily profit

### Weekly

- weekly sales summary
- weekly expenses
- weekly employee performance
- weekly profit

### Monthly

- monthly sales
- monthly customer credit balance
- monthly expenses
- monthly bills
- employee payouts
- monthly profit

### Yearly

- yearly sales
- yearly profit
- best employee
- best-selling services
- yearly expense total

---

## 5.9.2 Employee reports

Employees must be able to view only their own performance.

### Employee report sections

- today earnings
- this week earnings
- this month earnings
- this year earnings
- number of services completed
- recent activity

---

## 5.10 Profile management

Employees should be able to update limited personal information.

### Features

- update profile picture
- change password
- update limited personal details if allowed

# 6. Non-functional requirements

## 6.1 Mobile responsiveness

This is one of the most important requirements.

### Required behavior

- employee UI must be mobile-first
- admin dashboard must also work on mobile
- buttons should be large and easy to tap
- forms should be simple and readable on small screens

---

## 6.2 Usability

- easy navigation
- minimal steps for entering a sale
- clear financial summaries
- clean and modern interface

---

## 6.3 Security

- hashed passwords
- role-based access
- restricted permissions for employees
- protected admin routes

---

## 6.4 Performance

- quick loading dashboard
- fast search for customers and services
- optimized reports for daily business usage

---

## 6.5 Data integrity

- automatic timestamps for transactions
- keep historical sale values unchanged
- prevent unauthorized editing of financial records

---

# 7. Business rules

## Rule 1

Every employee has a configurable commission percentage.

## Rule 2

The owner share is automatically calculated from the remaining percentage.

## Rule 3

Service prices are controlled by admin.

## Rule 4

Employees cannot modify service prices unless admin allows it.

## Rule 5

When a sale is recorded, price and commission details must be saved as snapshots.

## Rule 6

If payment is delayed, the sale must be stored as credit.

## Rule 7

Expenses recorded by employees must be linked to the employee who recorded them.

## Rule 8

Monthly profit must consider:

- sales income
- employee earnings
- business expenses
- owner monthly bills
- credit amounts if using cash-based reporting

---

# 8. Suggested modules

1. Authentication Module
2. Employee Management Module

3. Service Package Module
4. Customer Management Module
5. Sales Transaction Module
6. Credit Management Module
7. Expense Management Module
8. Payee / Debtor Module
9. Reporting & Analytics Module
10. Profile Management Module

---

# 9. Recommended database structure

Here is a good starting structure.

## 9.1 users

- id
- username
- password_hash
- role
- employee_id nullable
- status
- created_at
- updated_at

## 9.2 employees

- id
- full_name
- mobile
- join_date
- profile_picture
- status
- created_at
- updated_at

## 9.3 employee_commissions

- id
- employee_id
- employee_percent
- owner_percent
- effective_from
- effective_to nullable
- created_at

## 9.4 services

- id

- service_name
- price
- description nullable
- is_active
- created_at
- updated_at

## 9.5 customers

- id
- customer_name
- mobile nullable
- notes nullable
- created_at
- updated_at

## 9.6 sales

- id
- invoice_no
- customer_id
- employee_id
- service_id
- service_name_snapshot
- service_price_snapshot
- employee_percent_snapshot
- owner_percent_snapshot
- employee_amount
- owner_amount
- payment_status
- paid_amount
- due_amount
- created_by
- created_at

## 9.7 credit_payments

- id
- sale_id
- customer_id
- amount_paid
- paid_at
- recorded_by
- note

## 9.8 payees

- id
- name
- type
- mobile nullable

- notes nullable
- is_active
- created_at

## 9.9 expense_categories

- id
- category_name
- category_type
- created_at

## 9.10 expenses

- id
- category_id
- payee_id nullable
- amount
- note
- recorded_by
- paid_by
- expense_date
- created_at

## 9.11 monthly_bills

- id
- bill_type
- amount
- bill_month
- due_date
- paid_date nullable
- status
- note

---

# 10. Basic ER relationship idea

- one employee can have many sales
- one employee can have many commission history records
- one customer can have many sales
- one service can belong to many sales
- one sale can have many credit payments
- one payee can have many expense records
- one user can record many sales and expenses

---

# 11. Suggested screen structure

## Admin screens

- Login
- Dashboard
- Employees
- Commission Settings
- Services
- Customers
- Sales History
- Credit Customers
- Expenses
- Payees / Debtors
- Bills
- Reports
- Profile / Settings

## Employee screens

- Login
- Dashboard
- New Sale
- Customers
- Credit Records
- My Earnings
- Record Expense
- My Profile

---

# 12. Development priority plan

## Phase 1

- login
- employee management
- service management
- customer management
- sales entry
- commission calculation
- employee earnings view

## Phase 2

- credit management
- expense recording
- payees management
- owner reports

## Phase 3

- monthly bills
- yearly reports
- advanced analytics
- export / print
- notifications

---

# 13. Important design recommendation

For this system, the **employee side should be designed first as mobile-first**, because daily usage happens mostly through phones. The admin side should also be responsive, but the employee experience must be the easiest and fastest part of the whole product.

---

# 14. Short professional summary

This salon management system will provide a fully customizable and mobile-friendly platform to manage salon services, employee commission sharing, customer records, credit handling, supplier/bank payments, monthly bills, and business profit reporting. The system will support role-based access for owners and employees, ensuring secure operation and accurate financial tracking.

# Tech Stack

**My suggestion:**
**Spring Boot + Next.js + MySQL** is strong enough for this salon system and also professional enough to scale later.

# Why this stack fits your project

## 1. Spring Boot

Very good for your backend because you need:

- authentication and role-based access
- strong business logic
- commission calculations
- credit handling
- expense tracking
- reporting APIs
- secure structure for admin/employee permissions

Spring Boot is especially good when the system has a lot of **business rules**.

Your salon system has many rules like:

- employee commission %
- owner share %
- editable service templates
- credit customers
- expenses by employee vs owner
- monthly profit calculation

That kind of logic is handled very well in Spring Boot.

---

## 2. Next.js

Also a good choice for frontend because:

- you can build a modern admin panel
- responsive mobile UI is easy with Tailwind
- employee mobile screens can feel app-like
- you can structure components cleanly
- form validation and dashboard building is very nice

For this project, you are not building just a simple website.
You are building a **business dashboard + mobile-friendly employee interface**.

Next.js is very suitable for that.

---

## 3. MySQL

MySQL is completely fine for this system.

You need relational data like:

- employees
- customers
- services
- sales
- expenses
- payees
- bills
- credit payments

That is classic relational data, so MySQL works well.

---

# My honest recommendation on your stack

## Recommended stack

- **Frontend:** Next.js

- **UI:** Tailwind CSS + shadcn/ui
- **Backend:** Spring Boot
- **Database:** MySQL
- **Auth:** JWT + refresh token
- **ORM:** Spring Data JPA / Hibernate
- **API docs:** OpenAPI / Swagger
- **Frontend API client:** Orval + TanStack Query
- **Validation backend:** Jakarta Validation
- **Validation frontend:** React Hook Form + Zod

That is a very modern and clean stack.

---

# Best architecture for your project

## Frontend

**Next.js App Router**

- admin dashboard
- employee mobile dashboard
- login page
- service entry page
- reports page
- expense page
- customer page

Use:

- **Tailwind CSS**
- **shadcn/ui**
- **React Hook Form**
- **Zod**
- **TanStack Query**
- **Orval**

This gives you:

- reusable components
- strong forms
- good API handling
- cleaner codebase

---

## Backend

**Spring Boot REST API**
Modules:

- auth module

- employee module
- customer module
- service module
- sales module
- expense module
- reports module
- profile module

Use:

- Spring Web
- Spring Security
- Spring Data JPA
- MySQL Driver
- Validation
- springdoc-openapi

---

## Database

**MySQL**
Use proper relational design with foreign keys.

---

# Why this stack is better than simpler stacks for this project

Because this is not just CRUD.

This system has:

- percentage-based commission logic
- snapshot-based sales history
- different user roles
- due payments
- real profit calculation
- employee performance reports
- mobile-first operational flow

That is where **Spring Boot backend discipline** helps a lot.

---

# What I would slightly improve in your idea

Your stack is already good.
I would just improve the development style.

## Better version of your stack

### Frontend

- Next.js 15/16
- TypeScript
- Tailwind CSS
- shadcn/ui
- TanStack Query
- React Hook Form
- Zod
- Orval

### Backend

- Spring Boot 3
- Java 17 or 21
- Spring Security
- JWT
- Spring Data JPA
- Hibernate
- Flyway for DB migrations
- OpenAPI / Swagger

### Database

- MySQL 8

### Deployment

- Frontend: Vercel
- Backend: VPS / Render / Railway / Docker server
- Database: MySQL on VPS / managed DB

---

# Important technical suggestions

## 1. Use TypeScript in Next.js

Do not build this in plain JS.

Because your system has many forms and business data types.

Use TypeScript so you can safely manage:

- employee
- service
- sale
- expense
- bill
- report DTOs

---

## 2. Use OpenAPI + Orval

Since you already like structured development, this is perfect.

Flow:

- Spring Boot exposes Swagger/OpenAPI spec
- Orval generates typed frontend API hooks
- Next.js consumes them with TanStack Query

This is a very pro setup.

---

## 3. Use DTOs properly in Spring Boot

Do not expose entity classes directly.

Use:

- request DTOs
- response DTOs
- mapper layer

Example:

- `CreateSaleRequest`
- `SaleResponse`
- `EmployeeSummaryResponse`
- `ProfitReportResponse`

That keeps your backend clean.

---

## 4. Use snapshots in sale records

Very important.

When sale happens, save:

- service name snapshot
- service price snapshot
- employee percent snapshot
- owner percent snapshot
- calculated employee amount
- calculated owner amount

Otherwise future edits will break old financial records.

---

## 5. Separate reports carefully

Create two kinds of financial reports:

### Operational report

- total services done
- employee performance
- credits
- expenses

### Profit report

- total sales
- owner share
- employee payouts
- expenses
- bills
- outstanding credits
- net profit

---

# Suggested backend package structure

Something like this:

```
com.salonms
 ├── auth
 ├── user
 ├── employee
 ├── customer
 ├── service
 ├── sale
 ├── expense
 ├── payee
 ├── report
 ├── common
 ├── config
 └── security
```

Inside each module:

- controller
- service
- dto
- entity
- repository
- mapper

This will keep the codebase organized.

---

# Suggested frontend app structure

```
src/
├── app/
│   ├── login
│   ├── admin
│   ├── employee
│   └── dashboard
├── components/
│   ├── ui
│   ├── forms
│   ├── cards
│   ├── charts
│   └── shared
├── features/
│   ├── auth
│   ├── employees
│   ├── customers
│   ├── services
│   ├── sales
│   ├── expenses
│   └── reports
├── lib/
```

```
│   ├── api
│   ├── utils
│   ├── auth
│   └── validations
```

That is scalable.

---

# My rating of your selected stack

For this project:

- **Spring Boot**: 9.5/10
- **Next.js**: 9/10
- **MySQL**: 9/10

Overall: **excellent choice**

---

# Only 3 warnings

## 1. Don’t make frontend and backend tightly coupled

Use API contracts properly.

## 2. Don’t mix employee UI and admin UI badly

They should feel different:

- employee = fast, simple, mobile-first
- admin = reports, management, analytics

## 3. Don’t skip DB migrations

Use **Flyway** from the beginning.

Very important.

---

# Final recommendation

Yes — **go with Spring Boot + Next.js + MySQL**.

It is a strong, professional, scalable stack for this salon management system.

# My final ideal stack for you:

- **Frontend:** Next.js + TypeScript + Tailwind + shadcn/ui
- **API consumption:** Orval + TanStack Query
- **Forms:** React Hook Form + Zod
- **Backend:** Spring Boot + Spring Security + JPA
- **Database:** MySQL
- **Docs:** Swagger / OpenAPI
- **Migrations:** Flyway
- **Auth:** JWT + refresh token

This is the stack I would recommend for your project.

Next, I can give you the **full project architecture**, including:
**frontend folders, backend packages, database tables, and development phases step by step**.